

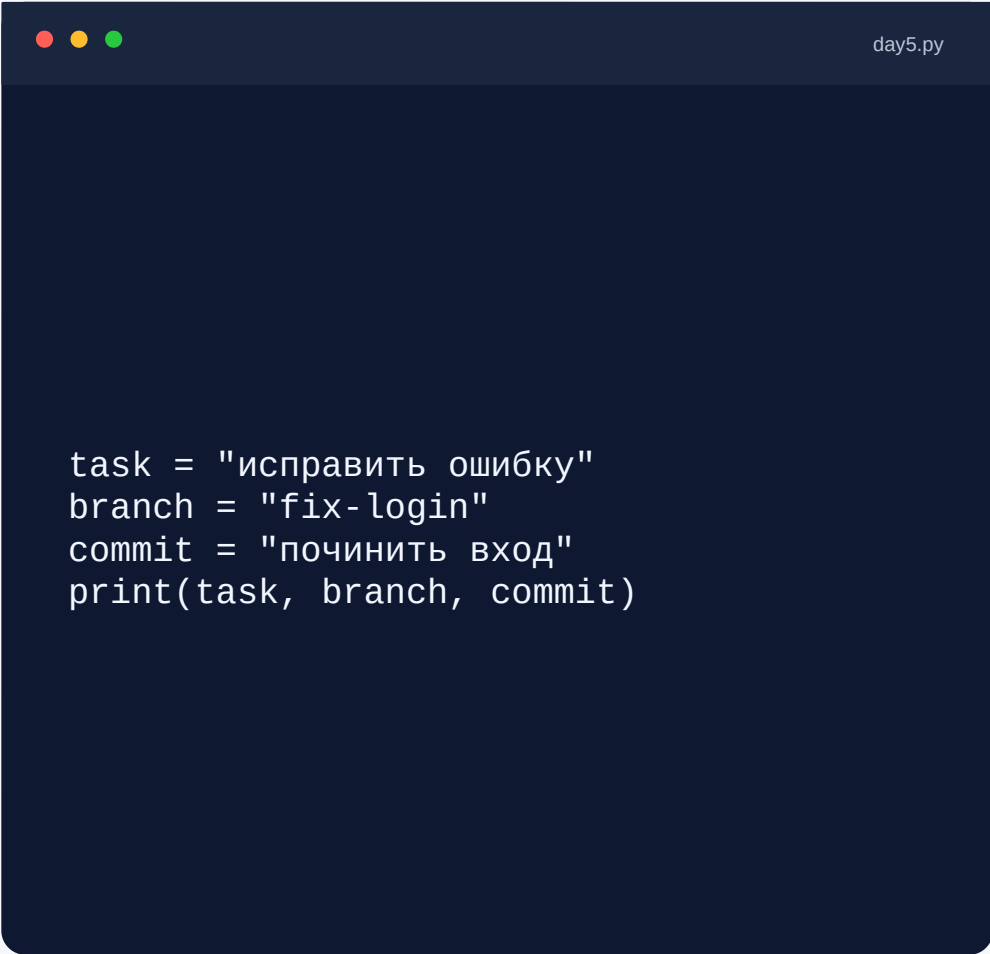
ФУНДАМЕНТ IT · ДЕНЬ 5 ИЗ 30

Как создают программы в команде

Знакомимся с кодом, задачами, версиями, ошибками и релизами.

Главная идея

Исходный код - текст, написанный программистом. Он хранится в репозитории вместе с историей изменений. Большая работа разбивается на небольшие задачи...



```
task = "исправить ошибку"
branch = "fix-login"
commit = "починить вход"
print(task, branch, commit)
```

План урока

Один блок теории - сразу практика и проверка.

1

Понять смысл

2

Выучить
термины

3

Разобрать код

4

Попробовать
самому

5

Собрать мини-
задачу

Как заниматься

Не читай как книгу. На практических слайдах остановись и попробуй решить сам.

Смысл темы простыми словами

Сначала бытовая идея, потом код.

Идея 1

Исходный код - текст, написанный программистом. Он хранится в репозитории вместе с историей изменений. Большая работа разбивается на небольшие задачи, которые можно проверить отдельно.

Идея 2

Разработчик создаёт отдельную ветку, меняет код, запускает проверки и отправляет изменения на review. После одобрения изменения объединяют с основной веткой.

Идея 3

Bug - поведение, которое не соответствует ожиданиям. Исправления и новые возможности накапливаются в версии продукта, а выпуск версии пользователям называется релизом.

Термины дня

Эти слова нужно узнавать в коде и документации.

source code

текст программы, написанный разработчиком

repository

хранилище проекта и истории его изменений

branch

изолированная линия изменений в репозитории

commit

зафиксированный снимок осмысленного изменения

bug

ошибка, из-за которой программа работает не так, как ожидается

release

подготовленная версия продукта, выпущенная пользователям

Код дня

Минимальный пример без лишней сложности.

```
task = "исправить ошибку"  
branch = "fix-login"  
commit = "починить вход"  
print(task, branch, commit)
```

Что ищем

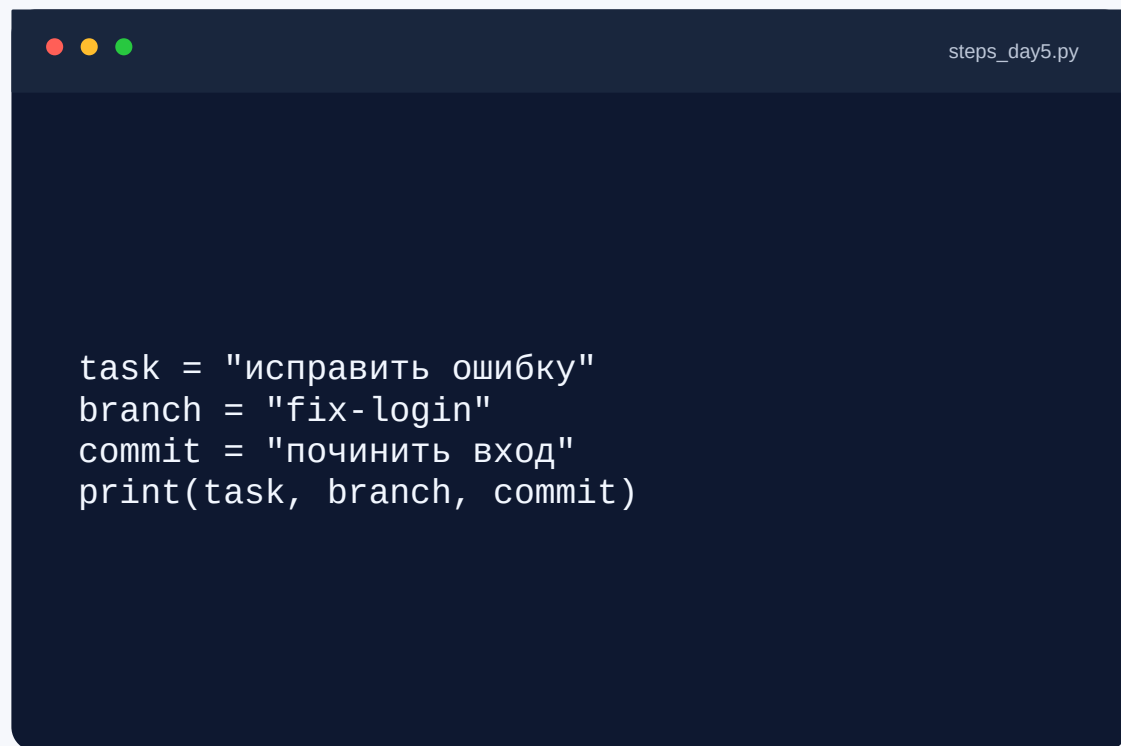
Где данные? Где действие? Где результат?

Главный вопрос

Можешь объяснить каждую строку обычным языком?

Разбор кода по шагам

Не переписываем вслепую.



```
task = "исправить ошибку"  
branch = "fix-login"  
commit = "починить вход"  
print(task, branch, commit)
```

- Прочитай строку сверху вниз.
- Назови данные, с которыми работает код.
- Объясни, какое действие выполняется.
- Проверь, какой результат должен получиться.

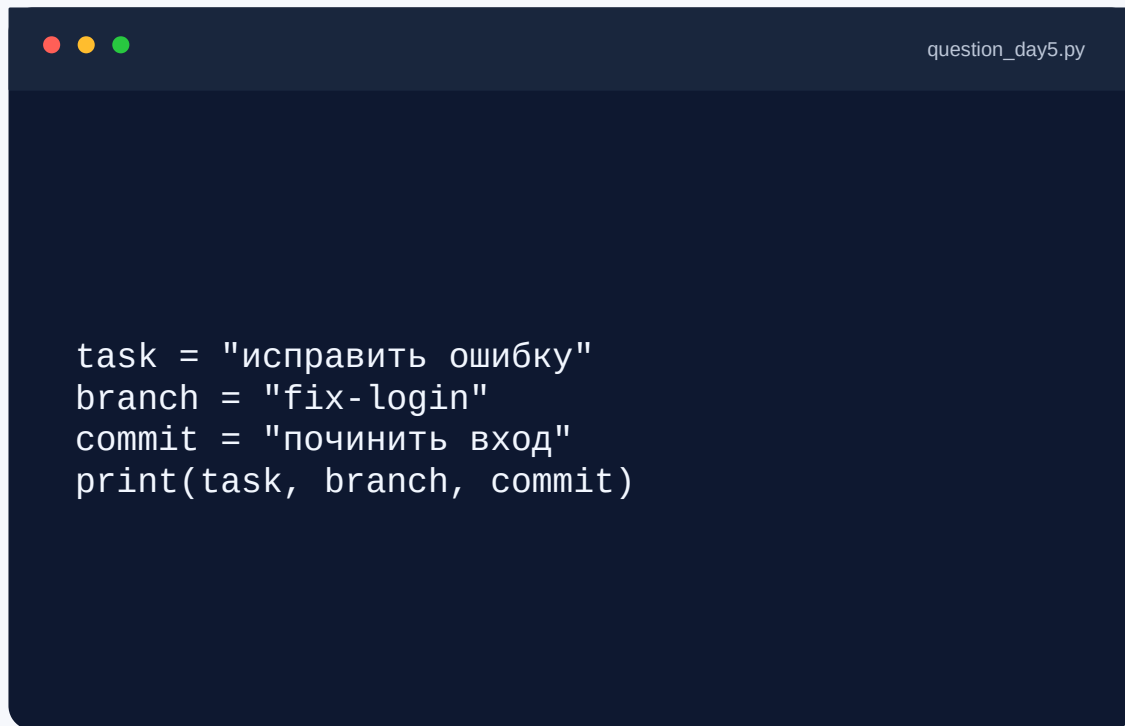


Важно

Если строка непонятна - остановись и переведи ее на обычный язык.

Практика: что произойдет?

Ответь без запуска кода.



```
task = "исправить ошибку"  
branch = "fix-login"  
commit = "починить вход"  
print(task, branch, commit)
```

Вопрос

Что программа сделает после запуска?

Подсказка

Ищи последовательность: входные данные -> действие -> результат.

Ответ и объяснение

Сравни со своим рассуждением.

Ожидаемая логика

Этот пример относится к теме: Как создают программы в команде. Знакомимся с кодом, задачами, версиями, ошибками и релизами.

Почему так?

Python выполняет инструкции по порядку. Поэтому важно читать код сверху вниз.

Проверка себя

Если можешь объяснить без подсказки - тема закрепляется.

Практика: сделай руками

Небольшое задание по теме дня.

Задание

Придумай одну feature-задачу и один bug.

Шаг 1

Создай простой файл или черновик.

Шаг 2

Напиши минимальный рабочий вариант.

Шаг 3

Запусти и проверь результат.

Мини-тест

Выбери ответ и объясни почему.

Вопрос

Зачем разработчик создаёт отдельную ветку?

A

Чтобы изолировать изменения до проверки и объединения

B

Чтобы ускорить интернет

C

Чтобы удалить историю проекта

D

Чтобы заменить операционную систему

Ответ мини-теста

Проверяем не память, а понимание.

Правильный ответ

А: Чтобы изолировать изменения до проверки и объединения

Объяснение

Ветка позволяет работать над задачей независимо и безопасно объединить результат после проверки.

Частая ошибка новичка

Лучше заранее увидеть, где обычно ломается.

Ошибка

Пытаться запомнить формулировку, но не понимать, где это видно в коде.

Как исправить

После каждого термина придумай свой пример и найди его в коде.

Правило

Термин считается выученным, когда ты можешь объяснить его на бытовом примере.

Самостоятельная мини-задача

Напиши маленький код или схему почти с нуля.

Условие

Придумай одну feature-задачу и один bug.

- Что программа получает?
- Что она должна сделать?
- Какой результат показать?
- Какие термины дня здесь применяются?

Разбор решения

После попытки сравни с этим подходом.

```
task = "исправить ошибку"  
branch = "fix-login"  
commit = "починить вход"  
print(task, branch, commit)
```

Не один вариант

Твой код может отличаться. Главное - он решает задачу и читается.

Проверка

Запусти пример и объясни результат вслух.

Задание со звездочкой

Немного сложнее, но без новой темы.

Условие

Расширь практику дня: добавь проверку ошибки, 2-3 тестовых примера и короткое объяснение для README.

Подсказка

Не усложняй. Сделай маленькое улучшение и обязательно проверь его на примере.

Итог дня

Что должно остаться после урока.

- объяснять тему "Как создают программы в команде" простыми словами
- узнавать главные термины дня
- читать короткий пример кода
- выполнять практику руками
- находить одну типовую ошибку
- формулировать следующий маленький шаг

Домашнее задание

Возьмите знакомое приложение и придумайте одну feature-задачу и один bug. Для каждого напишите ожидаемый результат одним предложением.